

Distributed Database Systems — Answer Key

Exam-oriented answer bank

This document provides structured answers for all questions in the distributed database question bank.

10-mark answers are written with definitions, explanations, working, advantages/limitations, and examples where appropriate.

5-mark answers are concise but complete enough for an exam answer.

10 Marks Answers

1. Explain Distributed Data Storage. Discuss different approaches for storing data in a distributed database.

Definition

Distributed Data Storage is the technique of storing data across multiple physical locations, computers, or database servers that are connected through a network. Although the data is physically distributed, the system provides users with a unified view of the database.

A distributed database may store different portions or copies of data at different sites depending on performance, availability, and reliability requirements.

Approaches to Distributed Data Storage

1. Data Fragmentation

Fragmentation divides a database relation into smaller logical parts called **fragments**.

There are three major types:

- **Horizontal fragmentation:** Rows are divided among different sites.
- **Vertical fragmentation:** Columns are divided among different sites.
- **Hybrid fragmentation:** A combination of horizontal and vertical fragmentation.

Example:

A university database can store student records according to location:

Pune Students	→ Pune Database Server
Delhi Students	→ Delhi Database Server
Mumbai Students	→ Mumbai Database Server

2. Data Replication

Replication stores copies of the same data at multiple sites.

Types include:

- **Full replication:** The complete database is copied to multiple sites.
- **Partial replication:** Only selected data is replicated.
- **Synchronous replication:** Copies are updated immediately.

- **Asynchronous replication:** Updates are propagated later.

Replication improves availability and read performance but increases storage and update-management overhead.

3. Data Allocation

Allocation determines **where each fragment or replica should be stored**.

The objective is to place frequently accessed data close to the users who need it while balancing storage and communication costs.

Advantages

- Improved availability
- Faster local access
- Better scalability
- Fault tolerance
- Reduced communication for local queries
- Load distribution

Challenges

- Maintaining consistency
- Network failures
- Distributed transaction management
- Security
- Increased system complexity

Conclusion

Distributed data storage improves scalability, availability, and performance by distributing data across multiple sites. Fragmentation, replication, and appropriate data allocation are the major techniques used to achieve these goals.

2. Explain Distributed Transactions and discuss the problems associated with executing transactions across multiple sites.

Definition

A **distributed transaction** is a transaction whose operations access or modify data stored at two or more sites in a distributed database.

For example, transferring money between accounts stored on different database servers may require an update at both sites.

```
Transaction T
|
+---- Site A: Debit Account
|
+---- Site B: Credit Account
```

The transaction must satisfy the **ACID properties** even though multiple sites are involved.

Working

A distributed transaction generally involves:

1. A transaction coordinator.
2. One or more participating database sites.
3. Execution of operations at participating sites.
4. Coordination to determine whether the transaction commits or aborts.

Problems in Distributed Transactions

1. Network Failure

Communication between sites may fail while a transaction is executing.

2. Site Failure

A database server may crash before completing the transaction.

3. Partial Commit

One site may commit while another site fails to commit. This can result in inconsistent data.

4. Concurrency Problems

Multiple distributed transactions may access the same data simultaneously, causing:

- Lost updates
- Dirty reads
- Inconsistent reads
- Incorrect results

5. Deadlock

Transactions running at different sites may wait for resources held by each other.

6. Communication Overhead

Messages must travel between sites, increasing transaction processing time.

7. Recovery Complexity

The system must recover consistently after site or network failures.

Example

Suppose a bank transfer moves ₹10,000 from Account A at Site 1 to Account B at Site 2.

The transaction must ensure:

```
Site 1 → Debit ₹10,000
Site 2 → Credit ₹10,000
```

If Site 1 commits but Site 2 fails, the transaction must be rolled back or recovered so that money is not lost.

Conclusion

Distributed transactions provide coordinated operations across multiple sites, but network failures, site failures, concurrency, deadlocks, and communication overhead make them more complex than local transactions.

3. Explain Commit Protocols in distributed databases. Discuss the Two-Phase Commit Protocol (2PC) with its phases.

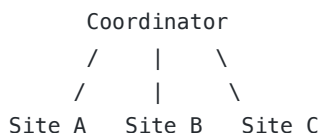
Definition

A **commit protocol** is a protocol used to coordinate multiple sites involved in a distributed transaction so that all participating sites reach the same final decision: **commit** or **abort**.

The main objective is to preserve the atomicity of distributed transactions.

Two-Phase Commit Protocol

The **Two-Phase Commit (2PC)** protocol uses a coordinator and multiple participants.



2PC consists of two phases.

Phase 1: Prepare / Voting Phase

1. The coordinator sends a **PREPARE** request to all participants.
2. Each participant performs the required transaction operations.
3. Each participant determines whether it can safely commit.
4. It sends either:
 - **YES / READY** if it can commit.
 - **NO / ABORT** if it cannot commit.

Phase 2: Commit / Abort Phase

If every participant votes YES:

Coordinator → COMMIT

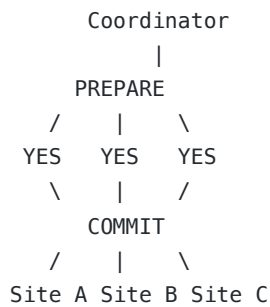
The participants commit the transaction and acknowledge the result.

If at least one participant votes NO:

Coordinator → ABORT

All participants roll back the transaction.

Example



Advantages

- Maintains atomicity
- Provides a coordinated commit decision
- Prevents different sites from independently committing or aborting under normal operation

Limitations

- Blocking can occur if the coordinator fails.
- Requires multiple network messages.
- Coordinator becomes an important point of dependency.
- Failure recovery is complex.

Conclusion

2PC is a fundamental distributed commit protocol that ensures all participating sites make a consistent commit or abort decision.

4. Describe the working of the Two-Phase Commit Protocol. Discuss its advantages, limitations and failure scenarios.

Working of 2PC

The Two-Phase Commit protocol coordinates a distributed transaction using a **coordinator** and several **participants**.

Phase 1: Prepare

1. The coordinator asks all participants to prepare.
2. Each participant executes the transaction locally.
3. Each participant writes necessary information to stable storage.
4. It votes:
 - YES if it is ready to commit.
 - NO if it cannot commit.

Phase 2: Decision

If all participants vote YES:

Coordinator → COMMIT

Otherwise:

Coordinator → ABORT

Participants then execute the decision and acknowledge it.

Failure Scenarios

Participant Failure

If a participant crashes before completing the transaction, the transaction may need to be aborted or recovered using its recorded state.

Coordinator Failure

If the coordinator fails after participants have voted YES but before they receive the final decision, participants may remain uncertain and block while waiting for the decision.

Network Failure

A communication failure may prevent messages from reaching participants. This can make participants wait for the final decision.

Advantages

- Ensures atomicity
- Provides a single global transaction decision
- Simple conceptual model
- Suitable for transactions involving multiple database sites

Limitations

- Blocking problem
- Communication overhead
- Coordinator dependency
- Recovery complexity
- Performance can decrease because participants may need to wait

Conclusion

2PC provides strong transaction consistency but can block during failures, especially when the coordinator becomes unavailable.

5. Explain Concurrency Control in Distributed Databases. Discuss why concurrency control is required and the problems it addresses.

Definition

Concurrency control is the set of techniques used to manage simultaneous execution of transactions so that database consistency is maintained.

In a distributed database, concurrency control is more complex because transactions may execute at different sites.

Need for Concurrency Control

Without concurrency control, simultaneous transactions may produce incorrect results.

It ensures:

- Isolation
- Consistency
- Serializable execution
- Correct handling of shared data
- Prevention of conflicting operations

Problems Addressed

1. Lost Update

Two transactions update the same data, and one update overwrites the other.

2. Dirty Read

A transaction reads data written by another transaction before that transaction commits.

3. Non-Repeatable Read

A transaction reads the same data twice and obtains different values because another transaction modified it.

4. Inconsistent Analysis

A transaction reads multiple values while another transaction is changing them.

5. Distributed Deadlock

Transactions at different sites wait for resources held by each other.

Common Techniques

Lock-Based Control

Transactions use shared and exclusive locks.

Timestamp Ordering

Transactions are ordered according to timestamps.

Optimistic Concurrency Control

Transactions execute without extensive locking and are validated before commit.

Conclusion

Concurrency control is essential for maintaining correctness when multiple transactions execute simultaneously across distributed sites.

6. Analyze the problems of maintaining data consistency during concurrent transactions in a distributed database. Suggest suitable concurrency-control mechanisms.

Data Consistency Problem

In a distributed database, multiple transactions can access replicated or fragmented data at different sites. Maintaining a consistent global state is therefore difficult.

Major Problems

1. Conflicting Updates

Two transactions may update the same data simultaneously.

2. Replicated Data

If a value has multiple replicas, updates must be propagated correctly.

3. Network Delays

Different sites may receive updates at different times.

4. Distributed Deadlocks

Transactions may hold locks at different sites and wait for each other.

5. Site Failures

A failure during an update can leave replicas temporarily inconsistent.

Suitable Mechanisms

1. Two-Phase Locking

Transactions acquire locks before accessing data and release them according to the locking protocol.

2. Timestamp Ordering

Each transaction receives a timestamp, and conflicting operations are ordered according to timestamps.

3. Optimistic Concurrency Control

Useful when conflicts are relatively infrequent. Transactions are validated before being committed.

4. Replication Control

Updates to replicated data must be coordinated so that replicas remain consistent.

5. Distributed Commit Protocols

2PC can ensure that a distributed transaction either commits consistently or aborts.

Conclusion

Distributed consistency requires coordination among sites. Locking, timestamp ordering, optimistic methods, replication control, and distributed commit protocols can be combined according to workload and consistency requirements.

7. Explain Distributed Query Processing. Discuss the major steps involved in processing a query in a distributed database.

Definition

Distributed Query Processing is the process of executing a database query when the required data is distributed across multiple database sites.

The objective is to produce the correct result while minimizing:

- Communication cost
- Processing cost
- Data transfer
- Response time

Major Steps

1. Query Decomposition

The original SQL query is analyzed and converted into smaller operations.

2. Data Localization

The system identifies where the required fragments or replicas are stored.

3. Global Query Optimization

Different execution strategies are considered and the most efficient plan is selected.

4. Local Query Processing

Each site executes the operations assigned to it.

5. Data Transfer

Intermediate results may be transferred between sites.

6. Result Assembly

The final results are combined and returned to the user.

Example

Suppose student records are distributed across Pune and Delhi.

A query asking for all students with a particular grade may require:

```
User Query
  ↓
Query Decomposition
  ↓
Pune Site + Delhi Site
  ↓
Local Processing
  ↓
Transfer Results
  ↓
Combine Results
  ↓
Final Result
```

Conclusion

Distributed query processing coordinates query execution across multiple sites and attempts to reduce communication and processing costs while producing the correct result.

8. Analyze how data distribution and communication cost affect distributed query processing. Explain suitable strategies for improving query performance.

Effect of Data Distribution

The physical location of data directly affects query performance.

If the required data is stored locally, processing is generally faster. If data must be transferred from remote sites, communication overhead increases.

Communication Cost

Communication cost includes:

- Number of messages
- Amount of data transferred
- Network latency
- Remote site processing delays

In many distributed systems, transferring large intermediate results can be more expensive than performing additional local computation.

Performance Improvement Strategies

1. Data Localization

Execute operations at the site where the required data is stored.

2. Selection Pushdown

Apply filtering operations as early as possible.

Instead of transferring all records:

```
Site → Transfer all rows → Filter
```

perform:

```
Site → Filter → Transfer only required rows
```

3. Projection Pushdown

Transfer only required columns instead of complete records.

4. Join Optimization

Choose an efficient location and method for joining distributed relations.

5. Replication

Frequently accessed data can be replicated closer to users.

6. Query Parallelism

Independent operations can execute simultaneously at multiple sites.

Conclusion

Data placement and communication cost have a major effect on distributed query performance. Query optimization should minimize unnecessary data transfer while exploiting local processing and parallel execution.

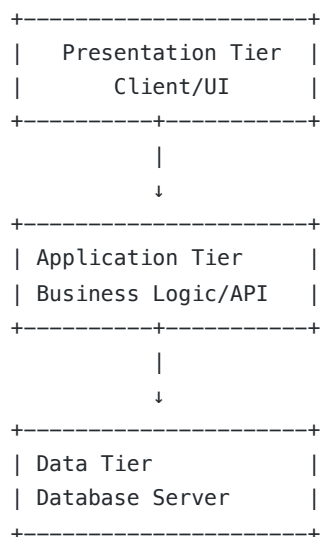
9. Explain the Three-Tier Client-Server Architecture with its components and working. Discuss its advantages.

Definition

A **Three-Tier Client-Server Architecture** divides an application into three logical layers:

1. Presentation Tier
2. Application / Business Logic Tier
3. Data Tier

Architecture



1. Presentation Tier

This is the user interface.

Examples:

- Web browser
- Mobile application

- Desktop application

It collects input and displays results.

2. Application Tier

This layer contains business rules and application logic.

It handles:

- Authentication
- Validation
- Transaction logic
- Authorization
- Communication with the database

3. Data Tier

This layer stores and manages data.

Examples:

- Relational databases
- Distributed databases
- Database management systems

Working

1. User sends a request through the presentation layer.
2. Application layer receives and validates the request.
3. Business logic is executed.
4. Application layer communicates with the database.
5. Database returns the required data.
6. Application layer processes the result.
7. Presentation layer displays the response.

Advantages

- Separation of concerns
- Easier maintenance
- Better scalability
- Improved security
- Reusability of business logic
- Easier database management
- Independent development of layers

Conclusion

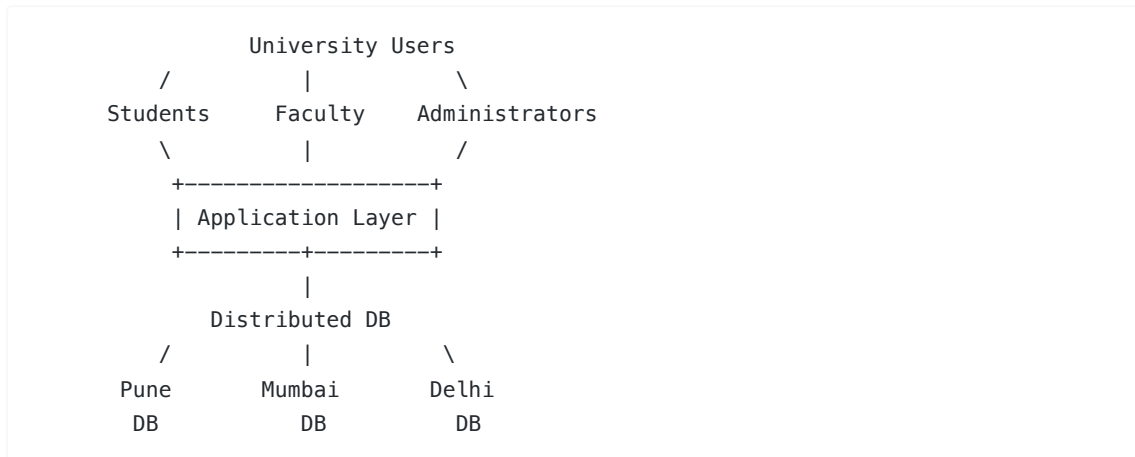
Three-tier architecture provides a modular structure that separates user interface, business logic, and data management, making applications easier to maintain and scale.

10. A university database is accessed by students, faculty and administrators from different locations. Design a suitable distributed

database architecture and explain data storage, transaction processing, concurrency control and query processing.

Proposed Architecture

A suitable architecture can use multiple regional database sites connected through a secure network.



1. Data Storage

Student, faculty, course, attendance, examination, and administrative data can be distributed using fragmentation.

For example:

- Student data can be horizontally fragmented by campus.
- Common reference data can be replicated.
- Frequently accessed data can have replicas at multiple sites.

2. Transaction Processing

Transactions involving multiple sites should use a distributed transaction coordinator.

For example, registering a student for a course may update:

- Student registration data
- Course enrollment data
- Seat availability

If multiple sites participate, a commit protocol such as 2PC can coordinate the final decision.

3. Concurrency Control

Concurrency control prevents conflicting transactions.

Suitable techniques include:

- Two-phase locking
- Timestamp ordering
- Optimistic concurrency control

For example, two students attempting to register for the final available seat should not both be allowed to successfully reserve it.

4. Query Processing

A distributed query processor should:

1. Decompose the query.
2. Identify relevant sites.
3. Select an efficient execution plan.
4. Execute local operations.
5. Transfer only necessary intermediate results.
6. Combine the results.

5. Security

Role-based access can be applied:

User	Example Access
Student	Own profile, courses, grades
Faculty	Courses, attendance, student records
Administrator	Administrative and management data

Advantages

- Local access is faster.
- System can scale across campuses.
- Failure at one site need not necessarily make all data unavailable.
- Workload can be distributed.
- Data can be placed close to users.

Conclusion

A distributed university database should combine fragmentation, selective replication, distributed transaction processing, concurrency control, and optimized query processing to provide reliable and efficient access across locations.

5 Marks Answers

21. What is distributed data storage?

Distributed data storage is the technique of storing database data across multiple physically separated sites connected through a network.

The data may be divided using **fragmentation** or copied using **replication**.

Advantages

- Improved availability
 - Better performance
 - Scalability
 - Fault tolerance
 - Reduced local access time
-

22. Define a distributed transaction with an example.

A **distributed transaction** is a transaction that accesses or modifies data stored at two or more database sites.

Example

A bank transfer may:

1. Debit ₹10,000 from an account at Site A.
2. Credit ₹10,000 to another account at Site B.

Both operations must be completed consistently for the transaction to commit.

23. What is the purpose of a commit protocol?

A **commit protocol** coordinates participating sites in a distributed transaction and ensures that they reach a consistent final decision.

Its main purpose is to maintain **atomicity**.

For example, using 2PC:

- If all participants are ready, the transaction commits.
 - If any participant cannot commit, the transaction aborts.
-

24. What are the two phases of 2PC?

The two phases of **Two-Phase Commit (2PC)** are:

1. Prepare / Voting Phase

The coordinator asks participants whether they are ready to commit. Participants respond with YES or NO.

2. Commit / Abort Phase

- If all participants vote YES → **COMMIT**
 - If any participant votes NO → **ABORT**
-

25. What is concurrency control?

Concurrency control is the process of managing simultaneous transactions so that database consistency and isolation are maintained.

It prevents problems such as:

- Lost updates
- Dirty reads
- Inconsistent results
- Non-repeatable reads

Common techniques include **locking, timestamp ordering, and optimistic concurrency control**.

26. State any five objectives of concurrency control.

Five objectives are:

1. Maintain database consistency.
 2. Ensure transaction isolation.
 3. Prevent lost updates.
 4. Prevent dirty reads.
 5. Ensure serializable or correct transaction execution.
-

27. What is distributed query processing?

Distributed query processing is the process of executing a query when the required data is stored at multiple database sites.

The system:

1. Decomposes the query.
2. Locates relevant data.
3. Optimizes the execution plan.
4. Executes operations at appropriate sites.
5. Combines the results.

The primary objective is to minimize processing and communication costs.

28. What is the role of communication cost in distributed query processing?

Communication cost is the cost associated with transferring queries, intermediate results, and data between database sites.

High communication cost can increase:

- Query execution time
- Network traffic
- System load

Therefore, distributed query optimization attempts to reduce unnecessary data transfer by using techniques such as **selection pushdown, projection pushdown, data localization, and efficient join strategies**.

29. Name and explain the three tiers of a three-tier architecture.

The three tiers are:

1. Presentation Tier

Provides the user interface and handles user interaction.

2. Application / Business Logic Tier

Contains business rules, validation, authentication, and application processing.

3. Data Tier

Stores and manages application data using a database server or database management system.

```
Presentation
  ↓
Application / Business Logic
  ↓
Data / Database
```

30. State any five advantages of a three-tier client-server architecture.

Five advantages are:

1. **Separation of concerns** — each layer has a specific responsibility.
 2. **Scalability** — individual layers can be scaled independently.
 3. **Maintainability** — changes can be isolated to a particular layer.
 4. **Security** — direct access to the database can be restricted.
 5. **Reusability** — business logic can be shared by multiple clients.
-

Repeated 5-Mark Questions

31. What is distributed data storage?

Distributed data storage stores database data across multiple physical sites connected through a network.

The data can be distributed using:

- **Fragmentation** — dividing data into smaller parts.
- **Replication** — maintaining copies at multiple sites.

It improves availability, scalability, and performance.

32. Define a distributed transaction with an example.

A distributed transaction is a transaction that involves data stored at multiple database sites.

Example: A bank transfer may debit an account at one site and credit another account at a different site. Both operations must be coordinated so that the transaction either commits completely or aborts.

33. What is the purpose of a commit protocol?

A commit protocol coordinates the participating sites of a distributed transaction and ensures that they make a consistent commit or abort decision.

The primary purpose is to maintain **transaction atomicity**.

Example: 2PC uses a prepare phase followed by a commit/abort phase.

34. What are the two phases of 2PC?

The two phases are:

1. **Prepare/Voting Phase:** Participants indicate whether they are ready to commit.
 2. **Commit/Abort Phase:** The coordinator instructs all participants to commit if everyone voted YES; otherwise, it instructs them to abort.
-

35. What is concurrency control?

Concurrency control manages the simultaneous execution of transactions to maintain database consistency and isolation.

It prevents conflicts such as:

- Lost updates
- Dirty reads
- Inconsistent reads

Common techniques include locking and timestamp ordering.

36. State any five objectives of concurrency control.

Five objectives are:

1. Maintain consistency.
 2. Ensure isolation.
 3. Prevent lost updates.
 4. Prevent dirty reads.
 5. Maintain correct/serializable transaction execution.
-

37. What is distributed query processing?

Distributed query processing executes database queries over data stored at multiple sites.

The query is decomposed, optimized, executed at appropriate sites, and the results are combined into the final answer.

Its main objective is to reduce communication and processing costs.

38. What is the role of communication cost in distributed query processing?

Communication cost represents the overhead of transferring data and messages between distributed database sites.

High communication cost can increase query response time.

Therefore, query processing attempts to:

- Reduce data transfer.
- Filter data before transmission.

- Transfer only required columns.
- Execute operations locally when possible.

39. Name and explain the three tiers of a three-tier architecture.

The three tiers are:

1. **Presentation Tier:** Provides the user interface.
2. **Application Tier:** Implements business logic and application processing.
3. **Data Tier:** Stores and manages database information.

The separation makes the system easier to maintain, secure, and scale.

40. State any five advantages of a three-tier client-server architecture.

Five advantages are:

1. Separation of presentation, business logic, and data.
 2. Improved maintainability.
 3. Better scalability.
 4. Improved security.
 5. Reusable business logic.
-

Quick Revision Sheet

Topic	Key Points
Distributed Data Storage	Fragmentation, replication, allocation
Distributed Transaction	Transaction across multiple sites
Commit Protocol	Coordinates commit/abort decision
2PC	Prepare/Vote → Commit/Abort
Concurrency Control	Maintains consistency during concurrent execution
Concurrency Problems	Lost update, dirty read, non-repeatable read, deadlock
Query Processing	Decomposition → Localization → Optimization → Execution → Result
Communication Cost	Network messages, data transfer, latency
Three-Tier Architecture	Presentation → Application → Data
University Distributed DB	Fragmentation + replication + distributed transactions + concurrency control + query optimization

Important Exam Keywords

Use these terms where relevant:

- **Distributed Database**
- **Fragmentation**
- **Horizontal Fragmentation**
- **Vertical Fragmentation**
- **Hybrid Fragmentation**
- **Replication**
- **Data Allocation**
- **Distributed Transaction**
- **Atomicity**
- **Two-Phase Commit (2PC)**
- **Coordinator**
- **Participant**
- **Prepare Phase**
- **Commit/Abort Phase**
- **Concurrency Control**
- **Two-Phase Locking**
- **Timestamp Ordering**
- **Optimistic Concurrency Control**
- **Distributed Deadlock**
- **Distributed Query Processing**
- **Query Decomposition**
- **Query Optimization**
- **Communication Cost**
- **Selection Pushdown**
- **Projection Pushdown**
- **Three-Tier Architecture**
- **Presentation Tier**
- **Application Tier**
- **Data Tier**